Linux network and host policy reference

Use with Phase 2 for FWR-04/FWT-04. These are proposed commands for APP1, not results. Stage F approval must identify the existing interface/backend, private values, exact permitted changes, validation and console rollback. Network/firewall path changes need the relevant C/D scope as well. Keep APP1's Proxmox console open; never execute this guide on the host, HP or another machine.

The private infrastructure map supplies LAB_NIC, LAB_APP1_IP/LAB_PREFIX_LENGTH, LAB_FW_RESTRICTED_IP, LAB_DC1_IP, LAB_TIME_IP and exact LAB_WIN1_IP/LAB_ADMIN_IP hosts. Replace every placeholder privately. APP1 has one Restricted zone NIC; a separately approved APP1-only overlay is the sole later exception. Inspect first and retain correct existing settings. Before every change save the current owning file, permissions, service/backend and full active ruleset into a new private recovery location; never overwrite an earlier copy.

Interface and backend inspection

Purpose and location: at APP1's Proxmox console under F approval, set a fixed Restricted address and permit only the application. The following is the conditional Ubuntu/Netplan/nftables profile. APP1 remains outside AD.

1. Match APP1's interface to its Proxmox MAC and inspect existing addresses/routes using the commands below. Save the current **/etc/netplan/** and firewall configuration into new approved private backup paths before editing; do not overwrite an earlier recovery copy. Record the actual renderer/backend in **02 / Windows and Linux Endpoint Controls**.

READ-ONLY - APP1 console, normal user for network views; sudo/root for protected firewall rules. These inspect addresses, routes, DNS and existing rules. No placeholders. Do not infer isolation from one view:

```
ip -br address
ip route
resolvectl status
sudo nft list ruleset
```

Check: a single zone NIC matches the approved Restricted bridge/tag/MAC record, with no home/production resolver or unintended forwarding rules. If a tool/backend differs, stop and verify the selected profile rather than installing another manager.

Netplan configuration and rollback

Target / approval: APP1 console, sudo/root, F-approved network scope. Expected configuration is shown below; preserve a private copy and console access before editing.

2. Using the approved privileged editor, edit the one existing Netplan YAML that owns the NIC. Preserve indentation and remove conflicting definitions from the approved source only; a second YAML file can merge with old settings. Replace all capitals with approved private values. Ubuntu's network guide explains the fields.

CONFIGURATION CHANGE - APP1, root-owned /etc/netplan configuration, privileged editor. File content only; LAB_NIC, addresses, prefix and gateways come from the private Network Zones record. IPv6 routing/autoconfiguration is not enabled in this baseline:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    LAB_NIC:
      dhcp4: false
      dhcp6: false
      accept-ra: false
      addresses: [LAB_APP1_IP/LAB_PREFIX_LENGTH]
      routes:
        - to: default
          via: LAB_FW_RESTRICTED_IP
      nameservers:
        addresses: [LAB_DC1_IP]
        search: [ad.lab.test]
```

Check before applying: the address belongs to Restricted, the gateway is its OPNsense interface and DNS is DC1. Do not paste the placeholders literally. Preserve any release-specific required fields.

CONFIGURATION CHANGE - APP1 console, sudo/root, F approval. Generate checks the merged syntax; try temporarily applies it and asks for confirmation. No placeholders after the file is filled:

```
sudo netplan generate
sudo netplan try --timeout 120
```

Check: confirm only after the address/route/resolver views match and lab DNS works. Netplan warns that rollback must be checked; on error or timeout verify the actual state at console and restore the saved file/settings before continuing.

Lab-only time

Target / approval: APP1 console, sudo/root, F approval. Save the prior service/drop-in and selected source so it can be restored at console if synchronization fails.

3. For the conditional Ubuntu **systemd-timesyncd** profile, preserve its current configuration. Confirm this service exists and no other time daemon/per-link DHCP NTP source overrides the plan; if a different daemon is selected, obtain its reviewed equivalent before continuing. FF-04 must permit the corresponding lab-only time dependency.

CONFIGURATION CHANGE - APP1 console, sudo/root, F approval. Create the selected service's configuration directory if absent; existing directory contents are preserved:

```
sudo install -d -o root -g root -m 0755 /etc/systemd/timesyncd.conf.d
```

Check: the directory is root-owned. In the approved editor save the following at **/etc/systemd/timesyncd.conf.d/10-lab-time.conf**, preserving an existing file before any edit.

CONFIGURATION CHANGE - APP1 root editor, 10-lab-time.conf. File content only. Replace LAB_TIME_IP with the exact approved lab time server; empty FallbackNTP removes the default fallback list:

```
[Time]
NTP=LAB_TIME_IP
FallbackNTP=
```

Check: only the selected lab server appears and permissions prevent ordinary-user edits. Do not use a public/corporate time source.

CONFIGURATION CHANGE followed by READ-ONLY - APP1 console, sudo/root for restart; conditional installed systemd-timesyncd only. Apply the reviewed time drop-in and inspect the selected source/clock. No placeholders after its NTP field is completed:

```
sudo systemctl restart systemd-timesyncd
timedatectl timesync-status
timedatectl status
```

Check: the intended lab source is selected, synchronization/timezone and observer UTC difference are recorded, and no fallback public server appears. If unavailable/unsynchronized, keep the dependency unresolved; do not add an Internet pool or change DC1 time. The systemd configuration reference describes global/per-link sources.

Nftables policy

Target / approval: APP1 console, sudo/root, exact F-approved host rules. Inspect the package service unit, current file ownership/includes and all base-chain priorities first. If the existing persistent file contains other managed tables, obtain a reviewed integration plan that preserves them; do not overwrite it with this single-table profile. Do not start or restart a service whose ExecStart/ExecStop would flush the full ruleset. The reviewed manual load below does not do that.

4. Prepare **/etc/nftables.conf** with the approved editor. The example below owns one fresh **inet lab_host** table; if that table already exists, review/reconcile it first instead of duplicating it. Preserve other tables. Do not use **flush ruleset**, which would erase other services' rules. Keep IP forwarding off and check both IPv4/IPv6 effective values at stage entry.

CONFIGURATION CHANGE - APP1, sudo/root editor, nftables only. Replace LAB_WIN1_IP and LAB_ADMIN_IP with exact approved IPv4 hosts. This table allows local HTTPS, restricts future overlay packets to HTTPS before established traffic, and drops forwarding. No SSH service is enabled by this example:

```
table inet lab_host {
    chain input {
        type filter hook input priority 10; policy drop;
        iifname "lo" accept
        ct state invalid drop
        iifname "tailscale0" tcp dport 443 counter accept
        iifname "tailscale0" counter drop
        ct state established,related accept
        ip saddr { LAB_WIN1_IP, LAB_ADMIN_IP } tcp dport 443 counter accept
    }
    chain forward {
        type filter hook forward priority 10; policy drop;
    }
}
```

Check: review the Ubuntu nftables behavior with the selected release: an accept in another base chain does not override a later drop here. The overlay rule still depends on Tailscale's exact source/destination grant in Phase 3. Approved egress remains restricted by OPNsense/physical isolation; this input example does not authorize Internet access.

Initial load and persistence

Target / approval: APP1 console, sudo/root, F approval. Use this exact profile only when the reviewed boot file owns lab_host alone and the service/includes preserve other tables. If lab_host already exists, use atomic replacement instead; never reload the table-creation file blindly.

CONFIGURATION CHANGE, preceded/followed by READ-ONLY - APP1 console, sudo/root. Check syntax first; only then load the new table and enable the package's persistent service. No replacements after the file is complete. Use this initial-load sequence once on the reviewed fresh table:

```
sudo nft -c -f /etc/nftables.conf
sudo nft -f /etc/nftables.conf
sudo systemctl enable nftables
sudo nft list table inet lab_host
```

Check: the input/forward drop policies and exact permits are active. Confirm persistence at an approved reboot, then test the HTTPS permit and denied listener once the application exists. If syntax/load fails, remain at console and restore the saved rules using the approved method; do not disable all filtering. Later edits need a reviewed table-replacement transaction, not repeated table creation.

Scoped atomic replacement

Target / approval: APP1 console, sudo/root, F approval naming the exact later change and rollback. Expected: one atomic replacement of lab_host, preserving all unrelated active tables and rules. If the persistent layout differs from the reviewed single-table profile, stop for its integration plan.

For a later approved change to this existing table, preserve the active rules and boot file first. Edit the complete **/etc/nftables.conf** to include only the intended narrow change. Save the following separate root-owned transaction as **/root/lab-host-replace.nft**; it is for an already existing table, not the boot configuration.

CONFIGURATION CHANGE - APP1 root editor, /root/lab-host-replace.nft. This file replaces only lab_host in one atomic nftables transaction, preserving other tables. The included boot file must contain the complete reviewed lab_host definition:

```
delete table inet lab_host
include "/etc/nftables.conf"
```

Check: /etc/nftables.conf still contains table creation only, without an unconditional delete that would fail at a fresh boot. If lab_host is absent, use the initial-load path instead.

CONFIGURATION CHANGE preceded/followed by READ-ONLY - APP1 console, sudo/root, exact approved rule scope. nft -c checks without applying; continue only after it succeeds:

```
sudo nft -c -f /root/lab-host-replace.nft
sudo nft -f /root/lab-host-replace.nft
sudo nft list table inet lab_host
```

Check: the specific changed rule and existing drop policies agree with the saved reviewed file; retest affected permit/deny paths. On failure restore the saved complete table/boot file at console using the same reviewed transaction, never flush other tables. This replacement method also handles the temporary mock and restored-client permits later in the manual.

Verification and console recovery

Target / approval: APP1 console, normal user for network views and sudo/root for protected rules; F-approved validation/reboot scope. Compare a saved full ruleset with the current ruleset to verify unrelated tables/rules are preserved.

READ-ONLY - APP1 console; sudo/root only for protected rules and service inspection. These checks add IPv6, forwarding and persistent service observations to the initial inventory; no placeholders:

```
ip -6 route
sysctl net.ipv4.ip_forward net.ipv6.conf.all.forwarding
sudo nft list ruleset
systemctl is-enabled nftables
systemctl status nftables --no-pager
```

Expected: IPv4 and IPv6 forwarding are 0; no unintended default/automatic IPv6 route or external DNS/time source; the exact lab_host input/forward drop policies and permits survive an approved reboot. Enabled/is-active status alone does not prove effective or persistent rules. Inspect per-interface IPv6 behavior and all chain interactions, including future Tailscale rules.

When the Phase 3 listener exists, pair the approved WIN1/admin HTTPS permit and one bounded denied request to a known listening lab service with a matching host-policy/log/counter observation; never probe protected networks. Missing listeners or scope mean NOT TESTED, not inferred denial. FWT-04 also needs the supported OS/build/update and ordinary-user denial/administrator control; nftables alone cannot pass the whole test. Record actual values, operator/date/version, private evidence IDs, second observations and verdicts in **02 > Windows and Linux Endpoint Controls** and **03 > Test Cases**. Log mismatches in **04 > Issue Log**.

If it fails: stay at console, keep external access disconnected and restore the saved network/firewall files only through the approved method. Verify the actual rollback and source/route/rule state before resuming; do not stack another manager or disable the host firewall.

For network rollback, restore the saved owning YAML and its permissions through the approved editor at console; validate the merged syntax and apply the approved recovery configuration, then repeat address, both route families, resolver and source checks. Do not assume a timed Netplan trial restored the actual state. For time rollback restore the saved drop-in/service choice and verify its effective source. For firewall rollback restore the saved complete lab_host boot definition and use the scoped transaction when the table exists; use initial load only when absent. Recheck persistence, unaffected tables and exact administrator/service access. Keep the external path disconnected throughout recovery.

Version and backend notes

This conditional Ubuntu Server profile uses existing Netplan/networkd, systemd-resolved, systemd-timesyncd and nftables. Record exact versions; NetworkManager, chrony or another backend needs a reviewed equivalent. Do not install or layer a competing manager. Keep configuration root-owned and resistant to ordinary-user edits.

Review Netplan YAML: inspect merged files, per-link/global/fallback DNS and IPv6 routes; no public/corporate fallback. Empty FallbackNTP removes compiled fallback; other drop-ins/per-link NTP must stay approved lab-only. No egress, SSH or Linux domain join is authorized. Future APP1-only overlay access needs exact enrolled-source/destination TCP 443 grants, physical HP policy and independent upstream isolation.